

Documentación Técnica: Proyecto "Mendacium"

1. Contexto del Proyecto

Descripción y Objetivo

Este proyecto consiste en el desarrollo de un videojuego móvil de deducción social al estilo "Werewolf" o "Mafia", programado en Kotlin para Android. El objetivo principal del juego es enfrentar a una mayoría desinformada (el bando de los Buenos) contra una minoría informada (los Impostores).

El objetivo académico es consolidar los conceptos de la Programación Orientada a Objetos (POO) aplicándolos a un entorno de desarrollo móvil moderno, demostrando cómo estructurar reglas de negocio complejas mediante polimorfismo y encapsulación.

Reglas Básicas del Juego

- **Roles Activos:** Al inicio de la partida, cada jugador recibe un rol secreto. En esta versión, se implementan cuatro roles fundamentales:
 - **Aldeano:** No tiene habilidades nocturnas. Su objetivo es deducir y votar de día.
 - **Impostor:** Despierta de noche para eliminar a un jugador.
 - **Médico:** Despierta de noche para proteger a un jugador (evitando que muera si es atacado por el Impostor).
 - **Vidente:** Despierta de noche para investigar a un jugador y descubrir su verdadero rol.
- **Fase de Noche:** Los jugadores "duermen" en la interfaz. El sistema cede el turno secuencialmente a los roles con habilidades nocturnas (Impostor, Médico, Vidente) para que ejecuten sus acciones en secreto.
- **Fase de Día:** El sistema calcula los resultados de la noche (evaluando ataques y protecciones). Se revela quién fue eliminado (si no fue protegido). Todos los jugadores vivos discuten y votan para "linchar" a un sospechoso.
- **Condición de Victoria:** El bando bueno gana si elimina a todos los impostores. Los impostores ganan si su número iguala o supera al de los jugadores restantes.

Público Objetivo

Estudiantes universitarios de Programación Orientada a Objetos 2 y entusiastas del desarrollo Android que deseen comprender cómo estructurar la lógica de un juego interactivo por turnos utilizando principios sólidos de ingeniería de software.

2. Arquitectura del Sistema

Arquitectura Utilizada: MVVM (Model-View-ViewModel)

Para este proyecto, se implementa la arquitectura **MVVM**, estándar en el desarrollo moderno de Android, especialmente al combinarla con **Jetpack Compose**.

Justificación

- **Separación de Responsabilidades:** MVVM aísla completamente la lógica matemática y estructural del juego (Modelo) de la interfaz gráfica (Vista).
- **Gestión del Estado:** En un juego de turnos con fases complejas, el estado cambia constantemente. El ViewModel mantiene este estado centralizado y sobrevive a los cambios de configuración del dispositivo (como rotaciones de pantalla).
- **Reactividad:** La Vista simplemente "observa" el estado expuesto por el ViewModel (mediante StateFlow) y reacciona automáticamente, redibujando la interfaz de Noche a Día sin intervención manual.

Diagrama Conceptual

1. **View (Vista):** Pantallas (ej. PantallaVotacion, PantallaTurnoNocturno). Envían eventos (seleccionar jugador, confirmar acción) al ViewModel.
2. **ViewModel (Gestor):** Recibe las interacciones, aplica las reglas de negocio llamando al Modelo y emite un nuevo estado inmutable.
3. **Model (Datos y Lógica Pura):** Clases Kotlin orientadas a objetos (Partida, Jugador, jerarquía de Rol) que contienen el motor del juego.

3. Modelado Orientado a Objetos

Clases Principales y Relaciones

- **Partida:** Clase central que compone el estado global. Maneja una colección de objetos Jugador y coordina la transición de fases.
- **Jugador:** Representa al usuario. Mantiene su estado vital (isAlive, isProtected) y contiene una instancia de Rol (Composición).
- **Rol:** Clase abstracta base que define el contrato para todos los personajes.
- **Voto:** Entidad de registro para la fase de linchamiento diurno.

Principios de POO Aplicados

- **Encapsulación:** Propiedades como isProtected o isAlive de la clase Jugador no pueden ser modificadas directamente por la interfaz. Solo la lógica de la Partida altera estos estados basándose en la resolución nocturna.

- **Herencia:** Se utiliza una clase sellada (sealed class) para el Rol. De esta base heredan clases específicas (RolImpostor, RolMedico, etc.), compartiendo atributos como nombre y bando.
- **Polimorfismo:** El método abstracto realizarAccion(objetivo: Jugador) es implementado de forma única por cada subclase. El sistema no necesita saber *qué* rol está actuando; simplemente invoca la acción, y el objeto responde según su naturaleza (el médico protege, el vidente inspecciona).

4. Estructura del Proyecto en Android Studio

Una organización limpia de paquetes facilita la escalabilidad:

- com.proyecto.impostor
 - model: Lógica de negocio independiente del framework Android (Jugador.kt, Rol.kt, MotorResolucion.kt).
 - viewmodel: Puente de comunicación (JuegoViewModel.kt).
 - ui:
 - theme: Sistema de diseño (colores nocturnos vs. diurnos).
 - screens: Funciones Composable de UI (LobbyScreen.kt, NightActionScreen.kt, VotingScreen.kt).
 - components: Elementos reutilizables (tarjetas de avatar, botones de confirmación).

5. Lógica del Juego (Flujo de Partida)

1. **Lobby:** Se registran los nombres. El creador inicia la partida.
2. **Repartición:** La clase Partida mezcla internamente los roles requeridos (ej. 1 Impostor, 1 Médico, 1 Vidente, N Aldeanos) y los asigna.
3. **Fase de Noche:**
 - El ViewModel establece el estado en NOCHE.
 - Se itera sobre los jugadores con habilidades activas.
 - Se presenta una pantalla a cada rol especial para elegir su objetivo, invocando jugador.rol.realizarAccion(objetivo).
4. **Resolución Nocturna:**
 - El motor del juego evalúa el choque de acciones. Si el objetivo del Impostor tiene isProtected == true (por obra del Médico), la muerte se cancela.
5. **Fase de Día:**
 - Se limpian los modificadores temporales (isProtected = false).
 - Se anuncia el resultado de la noche. Los jugadores debaten.
6. **Votación y Linchamiento:**
 - Los jugadores vivos emiten un voto.
 - Se cuenta el mayor número de votos y el jugador seleccionado es eliminado.
 - Se verifica la condición de victoria. Si nadie ha ganado, se vuelve al paso 3.

Anexos

Vidente

<https://www.figma.com/design/SaDVhGG7S6G64fDYHYjRjL/Vidente-view?node-id=0-1&t=KvdsS3NL35hPfFT9-1>

Impostor

<https://www.figma.com/design/PAzgK0WK0ahNdCJCsk6rvD/Impostor-view?t=KvdsS3NL35hPfFT9-1>

Médico

<https://www.figma.com/design/KhFsSBRLYBdgNO6f4kQFJi/Medico-view?t=KvdsS3NL35hPfFT9-1>

Aldeano

<https://www.figma.com/design/qIJbyYNRIz5utOg0PycCdI/Aldeano-view?node-id=0-1&t=A7IMUh5Zsk6OmMee-1>